

Name: Akshit Kumar

SAP ID: 590020087

Lab Experiment 8

File Handling & Exception Handling

1. Add few names, one name in each row, in "name.txt" file.

- Count number of names
- Count all names starting with vowel
- Find longest name

```
In [ ]: # Creating the file and adding the names
names_list = ["Akshit", "Vaibhav", "Deevyanshi", "Vikas", "Sna"]

with open("name.txt", "w") as file:
    for name in names_list:
        file.write(name + "\n")

# Reading the file
with open("name.txt", "r") as file:
    names = [line.strip() for line in file if line.strip()]

# Count number of names
total_count = len(names)

# Count names starting with vowel
vowels = "AEIOUaeiou"
vowel_count = sum(1 for name in names if name and name[0] in vowels)

# Find longest name
if names:
    longest = max(names, key=len)
else:
    longest = None

# Output
print("Total names:", total_count)
print("Names starting with vowel:", vowel_count)
print("Longest name:", longest)

Total names: 5
Starts with vowel: 0
Longest name: Deevyanshi
```

Explanation

- A list of names is created and written to a file named "name.txt", with each name on a new line.
- The file is opened in read mode, and all names are read into a list after removing newline characters using `strip()`.
- The total number of names is calculated using the `len()` function.
- To count names starting with a vowel, a string of vowels is defined.
 - A generator expression checks if the first letter of each name is a vowel.
- The longest name is found using the `max()` function with `key=len`.
- A condition is used to handle the case when the list is empty.
- Finally, all results are printed.

2. Store integers in a file:

- Find the maximum number
- Find average of all numbers
- Count number of numbers greater than 100

Code:

```
In [ ]: # Creating 'numbers.txt' and storing some integers
data = [5, 120, 8, 350, 42, 15, 200]

with open("numbers.txt", "w") as file:
    for num in data:
        file.write(str(num) + "\n")

# Reading the file
with open("numbers.txt", "r") as file:
    nums = [int(line.strip()) for line in file if line.strip()]

# Finding the maximum number
max_val = max(nums) if nums else None

# Finding average
average = sum(nums) / len(nums) if nums else 0

# Counting numbers greater than 100
count_gt_100 = sum(1 for n in nums if n > 100)

# Output
print("Max Number:", max_val)
print("Average:", round(average, 2))
print("Count > 100:", count_gt_100)

Max Number: 350
Average: 105.71
Count > 100: 3
```

Explanation

- A list of integers is created and written to a file named "numbers.txt", with each number on a new line.
- The file is opened in read mode, and each line is read and converted into an integer using `int()`.
- The maximum number is found using the `max()` function.
 - A condition is used to avoid errors if the list is empty.
- The average is calculated by dividing the sum of all numbers by the total count.
- A generator expression is used to count how many numbers are greater than 100.
- Finally, the results are printed.

3. Assume a file city.txt with details of 5 cities.

Open file city.txt and read to:

- Display details of all cities
- Display city names with population more than 10 Lakhs
- Display sum of areas of all cities

```
In [ ]: # Creating 'city.txt'
city_data = [
    "Dehradun 5.78 308.20",
    "Delhi 190.0 1484.0",
    "Mumbai 184.0 603.4",
    "Nainital 0.41 11.73",
    "Bangalore 84.0 709.0"
]

with open("city.txt", "w") as file:
    for line in city_data:
        file.write(line + "\n")

# Reading the file
total_area = 0
high_pop_cities = []

print("---- All City Details ----")

with open("city.txt", "r") as file:
    for line in file:
        if line.strip():
            parts = line.split()

            if len(parts) == 3: # safety check
                name = parts[0]
                pop = float(parts[1])
                area = float(parts[2])

                print(f"City: {name:10} | Population: {pop:>6}L | Area: {area:>8} sq KM")

                if pop > 10:
                    high_pop_cities.append(name)

                total_area += area

print("\n--- Cities with Population > 10 Lakhs ---")
print(", ".join(high_pop_cities))

print("\n--- Sum of Areas ----")
print(f"Total Area: {total_area:.2f} sq KM")

---- All City Details ----
City: Dehradun | Population: 5.78L | Area: 308.2 sq KM
City: Delhi | Population: 190.0L | Area: 1484.0 sq KM
City: Mumbai | Population: 184.0L | Area: 603.4 sq KM
City: Nainital | Population: 0.41L | Area: 11.73 sq KM
City: Bangalore | Population: 84.0L | Area: 709.0 sq KM

--- Cities with Population > 10 Lakhs ---
Delhi, Mumbai, Bangalore

--- Sum of Areas ---
Total Area: 3116.33 sq KM
```

Explanation

- A file named "city.txt" is created and city details are written into it.
 - Each line contains city name, population (in lakhs), and area.
- The file is opened in read mode to process the data.
- Each line is read and split into three parts: name, population, and area.
- The population and area values are converted into float for calculations.
- All city details are displayed in a formatted manner.
- Cities with population greater than 10 lakhs are identified and stored in a list.
- The total area is calculated by adding areas of all cities.
- Finally, the list of cities with high population and the total area are printed.

4. Input two values from user where the first line contains N, the number of test cases. The next N lines contain the space separated values of a and b. Perform integer division and print a/b. Handle exception in case of ZeroDivisionError or ValueError.

Code:

```
In [ ]: try:
    n = int(input("Enter number of test cases: "))
except ValueError as e:
    print(f"Error Code: {e}")
    n = 0

for _ in range(n):
    try:
        line = input().split()

        if len(line) < 2:
            raise ValueError("Insufficient values")

        a = int(line[0])
        b = int(line[1])

        print(a // b)

    except ZeroDivisionError as e:
        print(f"Error Code: {e}")
    except ValueError as e:
        print(f"Error Code: {e}")

Error Code: division by zero
Error Code: invalid literal for int() with base 10: '$'
```

Explanation

- The program first takes the number of test cases `n` as input.
 - Exception handling is used to catch invalid input using `ValueError`.
- A loop runs `n` times to process each test case.
- In each iteration, two values `a` and `b` are taken as input in a single line and split into a list.
- A check is performed to ensure that two values are provided.
- The values are converted into integers using `int()`.
- Integer division (`a // b`) is performed and the result is printed.
- Exception handling is used to manage errors:
 - `ZeroDivisionError`: when division by zero occurs
 - `ValueError`: when input is not a valid integer or insufficient values
- If any error occurs, an appropriate error message is displayed.

5. Create multiple suitable exceptions for a file handling.

Code:

```
In [ ]: class FileError(Exception):
    pass

class FileEmptyError(FileError):
    pass

class InvalidFormatError(FileError):
    pass

import os

def process_city_data(filename):
    try:
        if not os.path.exists(filename):
            raise FileNotFoundError(f"The file '{filename}' was not found.")

        if os.path.getsize(filename) == 0:
            raise FileEmptyError(f"'{filename}' is empty and cannot be processed.")

        with open(filename, 'r') as file:
            for line in file:
                if not line.strip(): # skip empty lines
                    continue

                parts = line.split()

                if len(parts) < 3:
                    raise InvalidFormatError(f"Line '{line.strip()}' is missing data.")

                print(f"Processing: {parts[0]}")

    except FileNotFoundError as e:
        print(f"System Error: {e}")
    except FileEmptyError as e:
        print(f"Data Error: {e}")
    except InvalidFormatError as e:
        print(f"Format Error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
    finally:
        print("File operation attempt finished.")
```

Explanation

- Custom exception classes are created:
 - `FileError` is the base class
 - `FileEmptyError` is raised when the file is empty
 - `InvalidFormatError` is raised when file content is incorrect
- The `os` module is used to check file existence and size.
- The function `process_city_data()` is defined to process file data.
- The program first checks if the file exists using `os.path.exists()`.
 - If not, a `FileNotFoundError` is raised.
- It then checks if the file is empty using `os.path.getsize()`.
 - If empty, a `FileEmptyError` is raised.
- The file is opened and read line by line.
- Each line is split into parts and validated.
 - If data is missing, an `InvalidFormatError` is raised.
- Different exceptions are handled using `try-except` blocks with appropriate messages.
- The `finally` block ensures a message is printed after execution, regardless of errors.

Name: Akshit Kumar

SAP ID: 590020087

Lab Experiment - 10

Data Analysis and Visualization

Question: Create numpy array to find sum of all elements in an array?

Code:

```
In [1]: import numpy as np

array = np.array([7, 12, 9, 4, 11, 25, 15])
print("Array: ", array)
print("Sum of all elements: ", np.sum(array))

Array: [ 7 12  9  4 11 25 15]
Sum of all elements: 83
```

Question: Create numpy array of (3, 3) dimension. Now find sum of all rows and columns individually. Also find 2nd maximum element in the array.

Code:

```
In [3]: matrix_3x3 = np.array([
    [9, 12, 15],
    [7, 12, 9],
    [4, 11, 25]
])

SumOfRows = np.sum(matrix_3x3, axis=1)
SumOfColumns = np.sum(matrix_3x3, axis=0)
SecondMaximum = np.unique(matrix_3x3.flatten())[-2]

print("3x3 Matrix ", matrix_3x3)
print("Sum of Rows: ", SumOfRows)
print("Sum of Columns: ", SumOfColumns)
print("Second Maximum element: ", SecondMaximum)

3x3 Matrix [[ 9 12 15]
 [ 7 12  9]
 [ 4 11 25]]
Sum of Rows: [36 28 40]
Sum of Columns: [20 35 49]
Second Maximum element: 15
```

Question: Perform Matrix multiplication of any 2 n*n matrices.

Code:

```
In [5]: MatrixA = np.array([
    [7, 12, 9],
    [11, 20, 25],
    [3, 9, 12]
])

MatrixB = np.array([
    [7, 12, 9],
    [11, 20, 25],
    [3, 9, 12]
])

productOfMatrix = MatrixA @ MatrixB

print("Matrix A: ", MatrixA)
print("Matrix B: ", MatrixB)
print("MatrixA x MatrixB", productOfMatrix)

Matrix A: [[ 7 12  9]
 [11 20 25]
 [ 3  9 12]]
Matrix B: [[ 7 12  9]
 [11 20 25]
 [ 3  9 12]]
MatrixA x MatrixB [[208 405 471]
 [372 757 899]
 [156 324 396]]
```

Question: Write a Pandas program to get the powers of an array values element-wise.

Code:

```
In [4]: import pandas as pd

baseValue = pd.Series([0, 1, 2, 3, 4, 5, 6], name = "base")
powerValue = pd.Series([0, 1, 2, 3, 4, 5, 6], name = "power")
poweredValues = baseValue.pow(powerValue)

print("Base Values: ", baseValue)
print("Power Values: ", poweredValues)

print("Powers of an array values element-wise: ", poweredValues)

Base Values: 0    0
1    1
2    2
3    3
4    4
5    5
6    6
Name: base, dtype: int64
Power Values: 0    1
1    1
2    4
3    27
4    256
5    3125
6    46656
dtype: int64
Powers of an array values element-wise: 0    1
1    1
2    4
3    27
4    256
5    3125
6    46656
dtype: int64
```

Question: Write a pandas program to get the first 3 rows of a given Dataframe.

Code:

```
In [7]: import numpy as np
import pandas as pd

examData = {
    "name": ["Anastasia", "Dima", "Katherine", "James", "Emily", "Michael", "Matthew", "Laura", "Kevin", "Jonas"],
    "score": [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    "attempts": [1, 3, 2, 5, 2, 3, 1, 1, 2, 1],
    "qualify": ["yes", "no", "yes", "no", "no", "yes", "yes", "no", "no", "yes"],
}

labels = ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j"]

examDataFrame = pd.DataFrame(examData, index=labels)

print("Complete DataFrame:")
print(examDataFrame)
print("First three rows:")
print(examDataFrame.head(3))

Complete DataFrame:
   name  score  attempts  qualify
a  Anastasia  12.5         1     yes
b      Dima    9.0         3     no
c  Katherine  16.5         2     yes
d     James   NaN         3     no
e     Emily    9.0         2     no
f   Michael  20.0         3     yes
g   Matthew  14.5         1     yes
h     Laura   NaN         1     no
i     Kevin    8.0         2     no
j     Jonas  19.0         1     yes
First three rows:
   name  score  attempts  qualify
a  Anastasia  12.5         1     yes
b      Dima    9.0         3     no
c  Katherine  16.5         2     yes
```